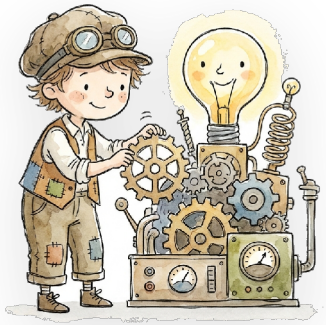
CALCULUS FOR OPTIMIZATION

Integral Calculus Foundations for ML



YOUR GUIDE

Let's get started.

The previous two chapters focused on the derivative, which involves taking a curve and determining how steeply it rises at a specific point. We can consider this the primary tool of *learning*, where differential calculus provides the gradient, and the chain rule allows backpropagation to carry that gradient through an entire network. In this chapter, we run this process in reverse. Instead of asking how fast a value is changing at a specific location, integration asks a different question. It might initially seem less useful; however, it asks: **"how much of this thing is there, in total, across a whole range?"**

We can see that this second question is not merely a leftover concept from a mathematics class. It is the operation hiding inside every probability we compute for a continuous quantity; however, it represents the chance a measurement lands in some range, the *average* value of a random variable, or the area a bell curve encloses. Deep learning trains on derivatives, but it *reasons* about the world using integrals. We will build this idea up from a visual representation and some deliberately small arithmetic, and by the end of this section, we can draw the entire concept on a napkin.

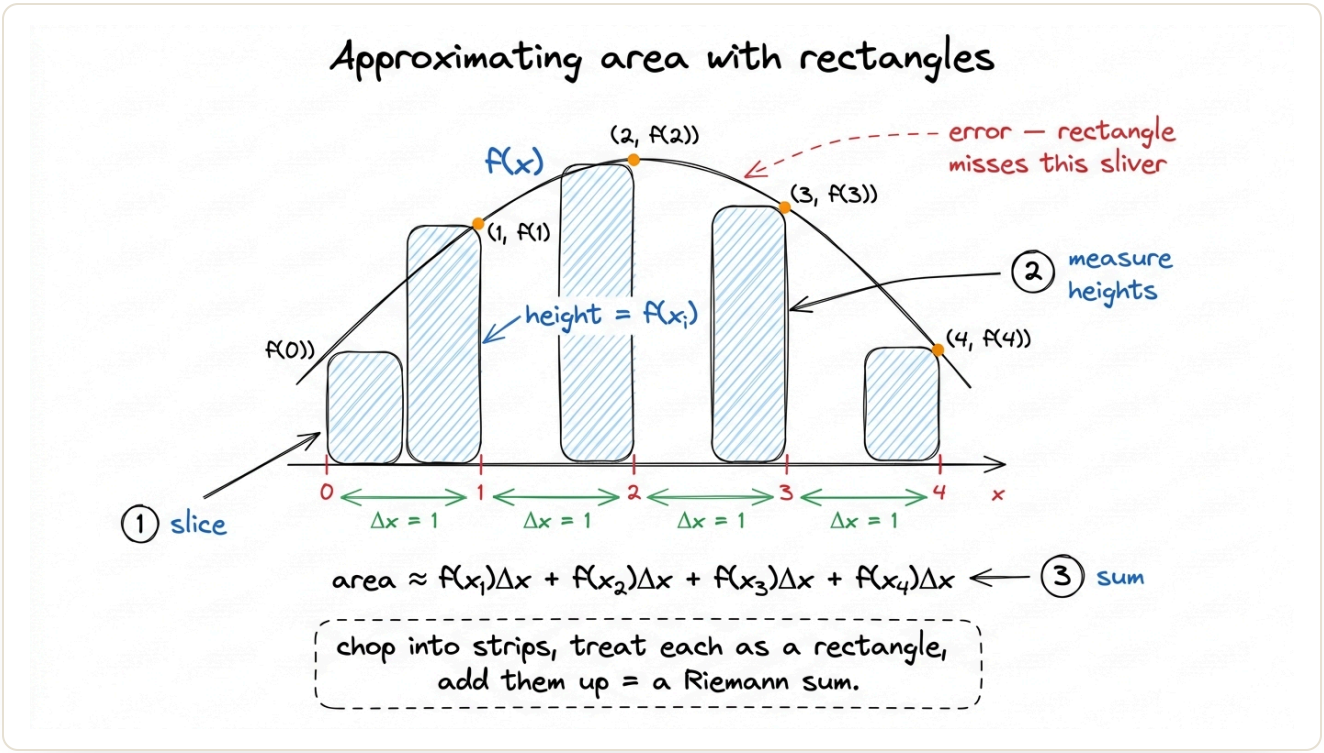
The puzzle: area under a wobbly curve

For example, we can consider a concrete problem. Suppose we have a function `f(x)`, and we plot it such that it forms a hill shape between `x = 0` and `x = 4`. We want to find the **area** trapped between that curve and the horizontal axis over that specific stretch.

If `f` were a straight line, this shape would be a triangle or a rectangle, and we could solve it with basic geometry. The difficulty arises because `f` curves; however, there is no clean formula for calculating the area under

a wobbling line. Therefore, we use an approximation technique, which is a fundamental approach in calculus. Specifically, **we approximate the curvy thing with a pile of simple things, and then we make the simple things smaller and smaller.**

The simple shape we use here is a rectangle. We slice the interval `[0, 4]` into a handful of vertical strips, and inside each strip, we treat the curve as if it is flat. This forms a rectangle whose height is `f` sampled somewhere within that strip. We then add up the rectangle areas. That sum is called a **Riemann sum**, and it forms the core mechanism of the integral.[1]



Slice the interval into strips, approximate each strip as a rectangle of height f(x_i), and sum the areas. That sum is a Riemann sum, and it approximates the true area.

We can make this fully concrete by using a function whose true area we already know, which allows us to observe the approximation as it converges. For example, we take `f(x) = x²` on the interval `[0, 3]`. We will use four rectangles, each with a width of `Δx = 0.75`, and we sample the height at the *right* edge of each strip.

The right edges are located at `x = 0.75, 1.5, 2.25, 3.0`. Their corresponding heights are the squares, which we compute as `0.5625, 2.25, 5.0625, 9.0`. Because each rectangle's area is its height times its width, we multiply the sum of the heights by `0.75`:

```
(0.5625 + 2.25 + 5.0625 + 9.0) × 0.75
= 16.875 × 0.75
= 12.66
```

As the output shows, four rectangles estimate the area as **12.66**. The exact answer, which we will confirm in a moment, is **9.0**. Our estimate is significantly higher than the true value; however, this is expected because right-edge rectangles on a rising curve extend above it, over-counting the area of every strip.[2] The solution to this inaccuracy is simply to use *more rectangles*.

Taking the slices to zero

Watch what happens as I shrink `Δx` and use more strips. With the same `f(x) = x²` on `[0, 3]`, right-edge sampling:

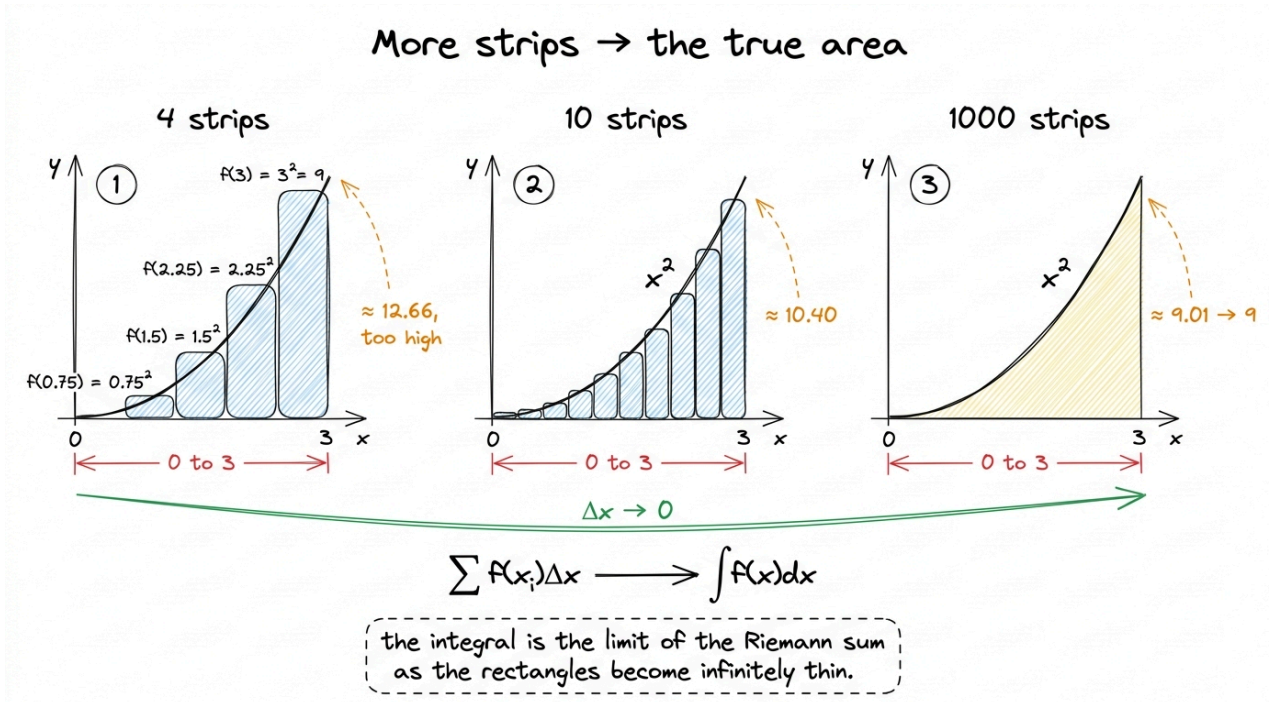
- **4 rectangles** (`Δx = 0.75`): area ≈ **12.66**

- **10 rectangles** (`Δx = 0.30`): area ≈ **10.40**
- **100 rectangles** (`Δx = 0.03`): area ≈ **9.14**
- **1000 rectangles** (`Δx = 0.003`): area ≈ **9.01**

As the output shows, the estimate converges steadily toward **9.0**. That limit represents the value the Riemann sum approaches as the strip width goes to zero; however, we formally call this the **definite integral**, and it receives its own specific notation:

$$\int_0^3 f(x)dx = \lim(\Delta x \to 0)\Sigma f(x_i)\Delta x$$

We can read the symbols sequentially to understand the operation. That elongated `S`, which is `∫`, represents a stretched "S" standing for **sum**. The `dx` notation indicates what `Δx` becomes when it shrinks to an infinitesimally thin slice. The `0` and `3` represent the left and right ends of the range. Therefore, `∫₀³ x² dx` specifies that we sum up `x² · dx` over infinitely many infinitesimally thin strips from 0 to 3. Ultimately, the integral is a Riemann sum evaluated at its limit.



As strip width shrinks toward zero, the Riemann sum converges to the exact area — and that limit is written as the definite integral ∫f(x)dx.

The shortcut nobody expects: the antiderivative

Summing a thousand rectangles by hand is a tedious process. A central result in calculus, known as the **Fundamental Theorem of Calculus**, demonstrates that we rarely need to perform this manual summation. It establishes that integration and differentiation are *inverse operations*. For

example, to find the area under `f`, we find a function `F` whose derivative is `f`, which we call an **antiderivative**. Consequently, the area from `a` to `b` is calculated simply as `F(b) - F(a)`.

For `f(x) = x²`, the power rule run backwards gives `F(x) = x³/3`, because the derivative of `x³/3` is `x²`. So:

$$\int_0^3 x^2 dx = F(3) - F(0) = 3^3/3 - 0^3/3 = 27/3 - 0 = 9$$

As the output shows, we compute exactly **9.0** in a single line, which matches the value our rectangle approximations were approaching.[3] The Riemann sum illustrates *what the integral means*, whereas the antiderivative shows us *how to compute it fast* when the function is well-behaved.

That's the pure-math core. Now the payoff — why an ML practitioner should carry this around.

Where integration actually shows up: probability

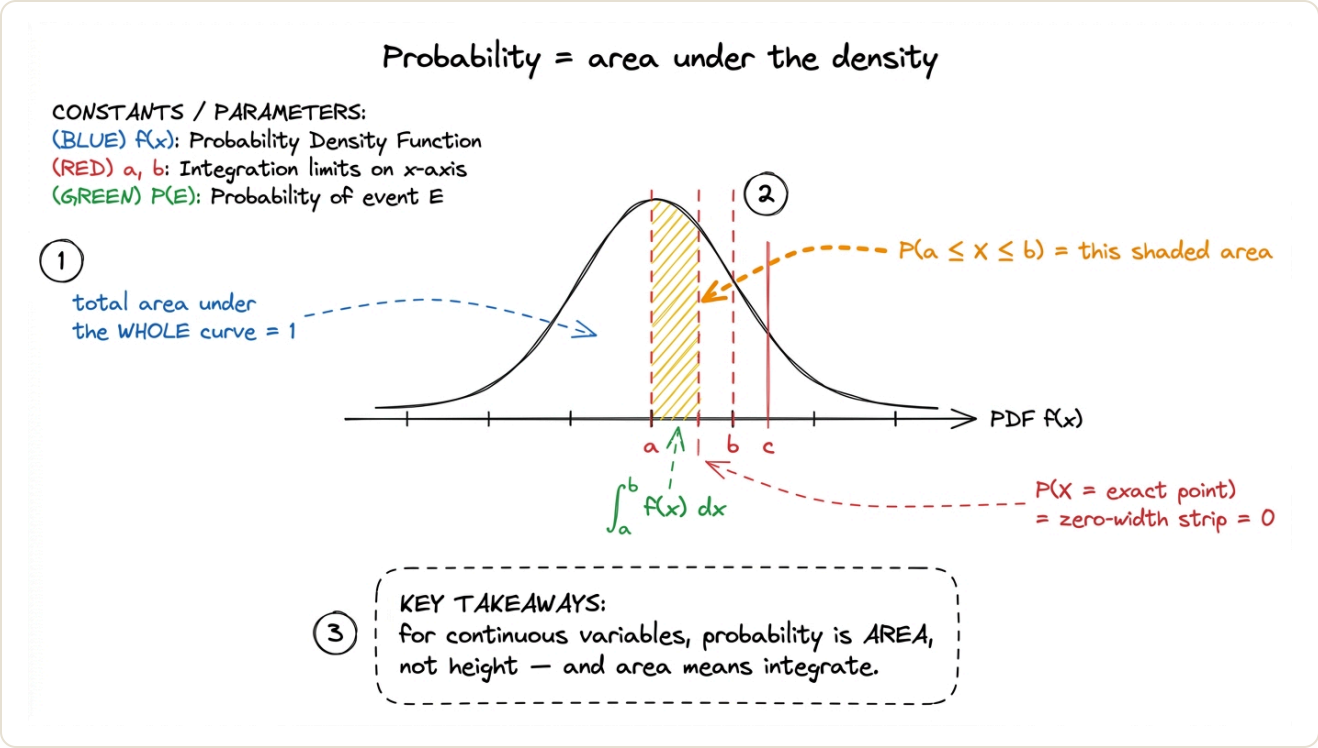
In probability distributions for ML, we discussed discrete distributions, where a random variable takes specific values, and each value has a specific probability. For a fair die, we have `P(X = 4) = 1/6`, and it makes sense to ask for the probability of a single exact outcome.

Continuous quantities change this behavior. If we ask for the probability that a person is *exactly* 180.000... cm tall, the answer is zero. There are infinitely many possible heights, however, so any single specific height has no probability mass. Continuous randomness is described instead by a **probability density function** (PDF), written as `f(x)`, and the rule changes accordingly. The probability is not the height of the curve; rather, it is the **area under it.** The chance that the value lands between `a` and `b` is exactly an integral:

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

Everything we know about area now transfers directly to this context, and two facts become immediately apparent. First, a probability can never exceed 1, so the *total* area under any PDF must equal exactly 1. This property is what makes it a valid distribution, yielding `∫ f(x) dx = 1` over the whole range. Second, the probability of any single exact point is `∫ₐᵃ`

`f(x) dx = 0`, representing a strip of zero width. This explains, however, why we only ever ask about *ranges* for continuous variables.

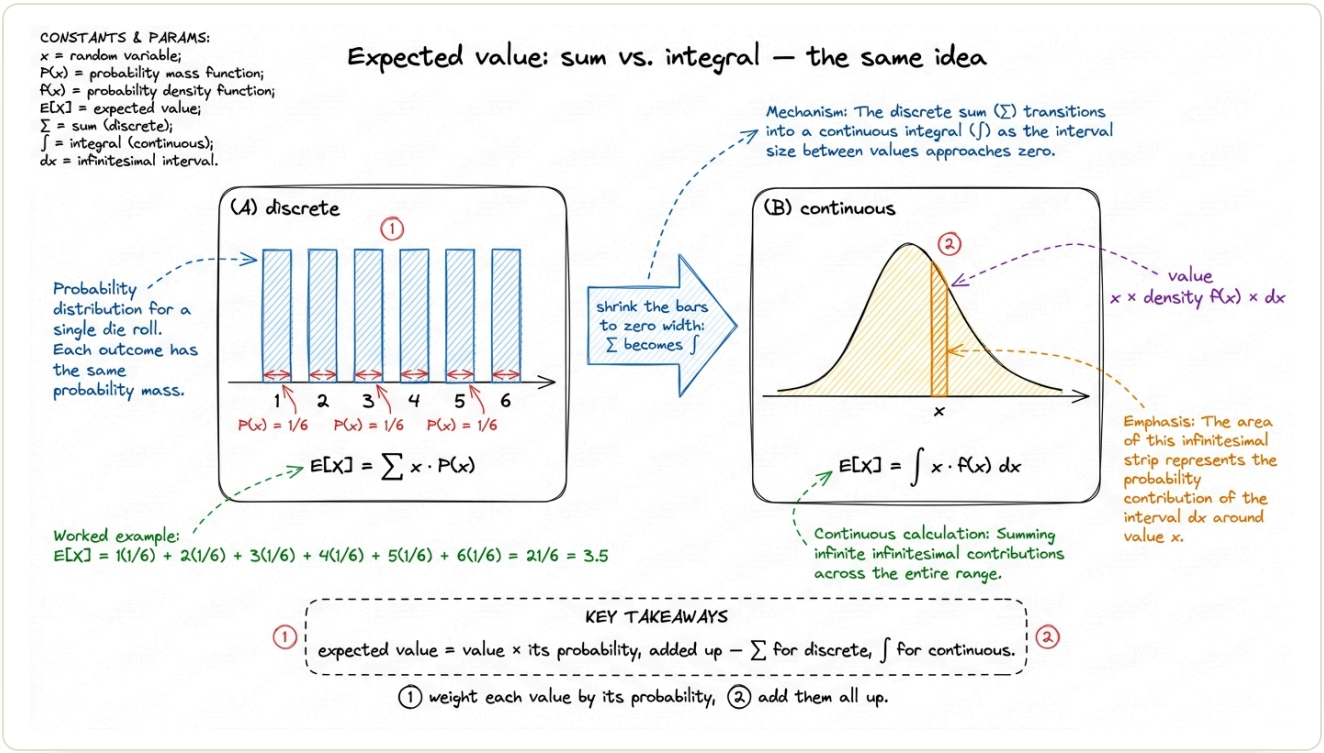


For a continuous distribution, the probability of landing in a range is the area under the density curve over that range — computed as a definite integral. The total area is always 1.

Note that this is an important concept, and it is the reason integration is included in a foundational machine learning book. Every time a model outputs a Gaussian distribution and we ask how confident it is that the true value is within a specific band, we compute `∫ f(x) dx`. Furthermore,

the normalizing constant that keeps a softmax or a Bayesian posterior a valid distribution is an integral that forces the total area to 1.

We should note a common pitfall that often causes confusion initially. Because `f(x)` is a *density* and not a probability, **the height of a PDF can be bigger than 1**, even though no probability ever can. The value on the vertical axis represents probability *per unit of x*, rather than the probability itself, so nothing forces this value below 1. Only the total *area* is constrained to 1. For example, we can consider a uniform distribution on the narrow interval `[0, 0.5]`. To enclose an area of exactly 1 over a base that is only `0.5` wide, the rectangle must stand `1 / 0.5 = 2` tall, which means `f(x) = 2` everywhere on that interval. A density of 2 is perfectly valid. The moment we squeeze the same unit of probability mass into a thinner range, the curve must spike upward to maintain an area of 1. Consequently, when we observe a Gaussian with a small `σ` extending past 1 on the y-axis, this is not an error; it is area conservation at work. We must always use the area, rather than the height.



A probability density can exceed 1 in height. Only the total area under the curve is fixed at 1, so squeezing the same mass into a narrower range forces the curve to rise above 1.

The other big one: expected value

The second place where integration applies to machine learning is the **expected value**. This is the long-run average of a random variable; however, it is also a highly important summary statistic in the field. For a

discrete variable, we weight each outcome by its probability and compute the sum:

$$E[X] = \Sigma x \cdot P(x)$$

For a *continuous* variable, the sum becomes — you can guess it now — an integral, with the density `f(x)` playing the role of the weight:

$$E[X] = \int x \cdot f(x) dx$$

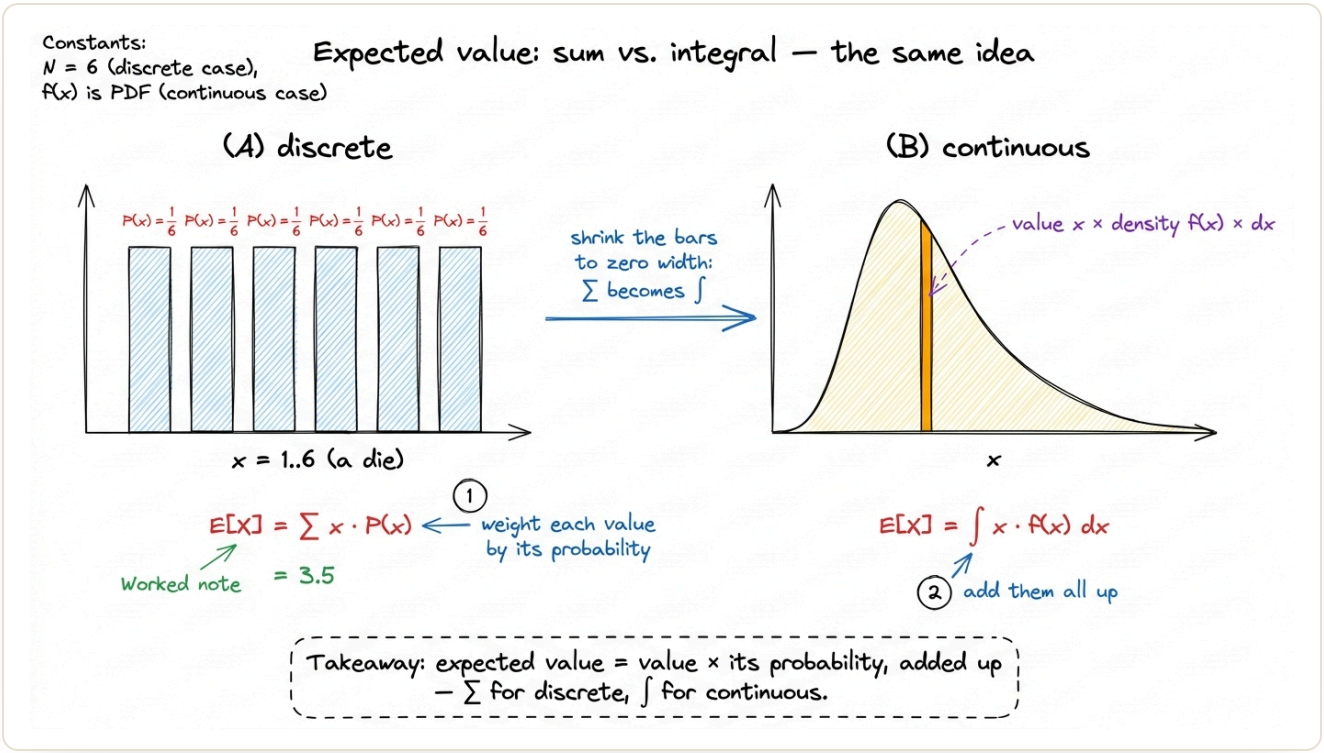
This represents the same concept viewed from two different perspectives, which is the value multiplied by its probability and added together. The discrete case uses `Σ`, whereas the continuous case uses `∫`. The Riemann-sum illustration serves as the bridge between them, however, because the integral *is* a sum of infinitely many infinitely thin pieces.

We can apply specific numbers to this example so that the formula becomes more concrete. For example, we can consider a **uniform distribution** on `[0, 2]`, which spreads its probability evenly, resulting in `f(x) = 0.5` everywhere on that interval (with base `2`, height `0.5`, and area `1`, making it a valid PDF). To determine the average value we expect to

draw, we compute the result by substituting into `E[X] = ∫ x · f(x) dx`, with the density `0.5` factored out to the front:

$$E[X] = \int_0^2 x \cdot 0.5 dx = 0.5 \cdot [x^2/2]_0^2 = 0.5 \cdot (4/2 - 0) = 0.5 \cdot 2 = 1.0$$

The answer is `1.0`, which is the exact center of the interval. This is precisely where we would expect the average of any value from 0 to 2, assuming all are equally likely, to be located. Note that this verification step is important to understand, because the integral did not produce an unexpected result. It instead recovered the intuitive answer. When a mathematical formula and our intuition align on a simple example, we can rely on that formula for more complex scenarios where our intuition is no longer sufficient.



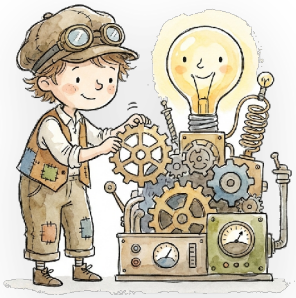
Expected value is 'value times probability, summed.' Discrete variables use a sum over outcomes; continuous variables use an integral over the density — the same operation with the rectangles shrunk to zero.

This concept is important because a **loss function** is, formally, an expected value. Specifically, it represents the *expected* error of our model over the true data distribution. We cannot integrate over the true distribution because we do not have access to it. In practice, we approximate that integral with a Riemann-style sum over our finite training batch; however, this is simply the mean loss we print during each

step.[4] For example, every time we write `loss.mean()`, we are computing a discrete approximation for a continuous expectation that we cannot calculate directly.

Note that the main takeaway from this section can be summarized as follows: **derivatives are how a model learns, but integrals are how it reasons about uncertainty.** The area under a curve, the probability of a range, the average of a random variable, and the expected loss are all represented by a single operation, `∫ f(x) dx`; however, this operation is simply a Riemann sum where we have shrunk the width of its rectangles to zero.

Now that our calculus toolkit is complete, covering both rates of change *and* accumulated totals, we are ready to transition from performing arithmetic on paper to implementing it in code. In the next section, we will set up the programming language that the remainder of this book uses: an introduction to Python for ML.



CHAPTER COMPLETE

Nicely done. You reached the end of this one.

NEXT CHAPTER Introduction to Python for ML →

0
Did this chapter land for you?

Mark chapter as done

Rate this chapter ★ ★ ★ ★ ★

What did you think of this chapter?

A takeaway, a question, a moment that clicked…

Be kind and specific. 0/2000

Post

No notes yet — be the first to share what you took away.

‹ The Chain Rule and Back…

Back to book

Introduction to Python … ›